Open in app ↗

Medium

Level Up Coding · Following

Member-only story

Writing Clean Git Commit Messages: Lessons From My Own Messy History

6 min read · Nov 4, 2025

Mathews Jose

Listen Share ••• More

An explanation of how to write clean, meaningful Git commits.

Photo by Roman Synkevych on Unsplash

I still remember the first time I scrolled through one of my own Git logs.
It was horrifying.

```
fix stuff
update again
final version
final-final-version (really this time)
ok now it works
```

I stared at those lines and felt like I was looking at cave paintings — a vague record of my past struggles, but absolutely no clue what they meant. I had no idea *what* I'd fixed, *why* it was broken, or *when* I'd stopped caring.

That's when it hit me:
Git isn't just a version control tool.
It's a communication tool.

And those commit messages? They're your code's journal entries — a story of how your project grew, improved, and occasionally fell apart at 2 a.m. before your first coffee.

Over time, I learned that writing **clean commit messages** isn't just about being "professional."
It's about empathy — for your teammates, for open-source contributors, and for your future self who will one day curse present-you for writing `update files`.

Here's what I've learned — painfully, but effectively — about writing commit messages that make your codebase readable, your teammates happy, and your future self grateful.

Why Clean Commit Messages Matter (and Why I Learned It the Hard Way)

When you're in the zone — refactoring a complex feature or fixing a tricky bug — it's tempting to just commit and move on. "I'll document later," we tell ourselves.

Spoiler: you won't.

A good commit message isn't just a log entry. It's a breadcrumb trail of intent.
Six months later, when your app mysteriously breaks and you're knee-deep in Git blame, your commit history becomes your lifeline.

Good commit messages help you:

- **Understand the "why"** behind changes (because "what" is visible in the code).
- **Collaborate effectively** with teammates — nobody wants to read diffs to guess intent.
- **Debug faster**, tracing bugs through a meaningful history.
- **Automate more easily**, since tools like *semantic-release* or *commitlint* rely on structured commit messages.

Think of your Git history as a **project diary**, not a trash can.
Your code might change, your docs might rot — but your Git history will always be there. Make it worth reading.

The Structure of a Good Commit Message

Most clean commit messages follow a simple but powerful pattern:

```
<type>: <short summary>
<optional body>
<optional footer>
```

Once you learn this rhythm, you'll wonder how you ever lived without it.

1. The Type

The **type** describes *what kind* of change you made.
Following the Conventional Commits standard makes this consistent and automation-friendly.

TypeMeaningfeatA new featurefixA bug fixdocsDocumentation onlystyleFormatting changes (no logic)refactorCode restructuring without behavior changetestAdding or updating testschoreMaintenance tasks, dependency updates, etc.

Example:

```
feat: add user profile page
```

Once your team starts using these consistently, you'll be able to scan Git history and instantly know what's what — no mental gymnastics required.

2. The Short Summary (a.k.a. The Subject Line)

Your subject line is your headline. It should tell the story at a glance.

Golden rules:

- Use **imperative mood** — say "add," "fix," "update," not "added" or "fixed."
- Keep it under **50 characters**.
- Don't end with a period.
- Avoid fluff like "update" or "stuff."

Examples:

✅ `fix: handle login button click on mobile`

❌ `fixed the issue where login button didn't work`

Think of it as a command:
"Git, please *add*, *fix*, or *refactor* this."
It reads beautifully when Git says things like "Apply patch" or "Merge branch."

3. The Body (Optional, But So Valuable)

This is where you explain *why* you made the change. The “what” is in the code; the “why” belongs here.

If you spent more than five minutes thinking about a change, the body is worth writing.

Guidelines:

- Explain *why*, not just *what*.
- Wrap text at around 72 characters per line.
- Mention design decisions, side effects, or limitations.

Example:

```
fix: resolve race condition in user session handling

The session middleware was being initialized twice due to concurrent
API requests. Added a mutex to ensure session consistency across
parallel calls.
```

That body text? It’s pure gold when someone (maybe you) debugs this six months later.

4. The Footer

The footer is for extra context — issue references, breaking changes, or links.

Example:

```
feat: implement new payment gateway integration
```

```
Closes #142
BREAKING CHANGE: The old PaymentService class has been replaced
by PaymentGatewayService.
```

This makes issue tracking and changelog generation seamless. Automation tools will thank you.

Real-Life Example: Before vs. After

Let's look at an actual transformation.

Before:

```
update files
```

After:

```
refactor: extract database config into separate module

Moved database connection settings from app.js to config/db.js to
improve maintainability and support future database migrations.
```

The difference? Night and day.
The second message tells a story — what happened and *why*. No guesswork required.

Best Practices for Writing Clean Commits

Here's how I gradually turned my Git logs from chaos into clarity.

1. Commit Small, Commit Often

Each commit should represent **one logical change**.
That makes it easier to review, revert, and understand.

Bad:

```
feat: add login, fix navbar, update footer
```

Good:

```
feat: add login feature
fix: correct navbar alignment
chore: update footer links
```

Small, atomic commits are your best debugging insurance policy.

2. Keep the Subject Short But Descriptive

Avoid filler words like “update” or “fix bug.”
Be specific about *what* changed.

Ask yourself:

> “If someone only reads this subject line in a list of 1,000 commits, will they understand what happened?”

If the answer is no, rewrite it.

3. Use the Body When It Adds Value

A commit message isn't a tweet.
It's okay to explain your reasoning, especially for complex changes or refactors.

Your future self — the one trying to remember why you made that database change — will silently thank you.

4. Reference Issues Clearly

Always link related tickets, issues, or tasks. It ties code to context.

Example:

```
fix: correct validation logic for user age (fixes #217)
```

This simple habit connects the dots for reviewers and future maintainers.

5. Clean Up Before You Merge

We've all seen commits like:

```
wip
fix again
okay really final
final-final
```

Before merging to `main`, **squash** or **rebase** those into meaningful commits.
Your teammates — and your Git history — will appreciate it.

Bonus: Automate Your Commit Hygiene

You don't have to rely on discipline alone.
Tools can help you enforce consistency automatically.

- **commitlint** → checks commit message formatting
- **husky** → runs Git hooks before commits
- **semantic-release** → automates changelogs and versioning

Example Husky hook:

```
#!/bin/sh
. "$(dirname "$0")/_/husky.sh"

npx commitlint --edit $1
```

With these in place, your team's commit messages stay clean — even on Monday mornings.

The Philosophy Behind It All

Writing good commits isn't about bureaucracy or process police.
It's about **empathy**.

Empathy for:

- Your teammates, who review your code.
- Your future self, who will debug your code.
- Your tools, which rely on consistent messages to automate workflows.

A clean Git history tells the story of your project's growth — one meaningful commit at a time.
It's your project's autobiography.

Final Thoughts

You don't have to nail this on day one.
Just start with one simple rule:

> "If someone reads this commit six months from now, will they understand it?"

If the answer is yes, you're already ahead of many developers.

Every time you replace a `fix stuff` with a clear, thoughtful message, you make your project — and yourself — a little more professional.

Clean commits are small acts of craftsmanship. They take seconds to write, but their value compounds for years.

TL;DR — The Commit Message Formula

```
<type>: <short summary>
<why/what explanation>
<references or breaking changes>
```

Example:

```
refactor: simplify authentication middleware

Removed redundant session checks and unified token validation
across endpoints.
Closes #98
```

If you found this helpful, follow for more developer experience lessons — not from textbooks, but from real scars earned in messy repos.

About the Author

***Mathews Jose** is a Software Engineer with over a decade of experience specializing in PHP, REST APIs, and event-driven architectures. He's passionate about building scalable backend systems and exploring how emerging technologies like AI can seamlessly integrate into modern software architecture.*

Connect with Mathews on LinkedIn

Programming | Git | Software Engineering | Developer Productivity | Clean Code

Following

Published in Level Up Coding

309K followers · Last published 1 day ago

Coding tutorials and news. The developer homepage gitconnected.com && skilled.dev && levelup.dev

Written by Mathews Jose

213 followers · 31 following

Software engineer sharing blending PHP, DDD, microservices, and AI exploration into practical, developer-friendly insights.

Responses (2)

See all responses